

90-Day Calisthenics Program Blueprint

Wellness App Content & Technical Implementation Guide

1. Executive Summary & App Architecture

This document serves as the complete operational blueprint for deploying a structured, progressive 90-day calisthenics workout plan within a consumer mobile fitness application. The curriculum is optimized to scale across varying user performance baselines while tracking progress, logs, and workout compliance.

Data Schema Integration Highlights:

- **Program Duration:** 90 Days divided into 3 macrocycles (Blocks) of 30 days each.
- **Progression Metric:** RPE (Rate of Perceived Exertion) and target volume increments.
- **Workout Frequency:** 4 Active days, 1 Flexibility/Mobility day, 2 Rest days per week.

2. 90-Day Macrocycle Architecture

To drive optimal athletic progression and prevent plateaus, the 90 days are split into three thematic architectural blocks. Each block demands updated UI copy, user check-ins, and distinct metrics.

Phase Block	Focus & Objectives	Tracking Focus
Block 1 (Days 1-30): Foundational Volume	Conditioning connective tissue, establishing form baselines, and building endurance via high reps.	Total completed repetitions & baseline test data.
Block 2 (Days 31-60): Strength Progression	Transitioning to eccentric loading, introducing mechanical disadvantages, and lowering reps.	Time Under Tension (TUT) and RPE metrics.
Block 3 (Days 61-90): Power & Mastery	Explosive movements, unilateral variants, and integrated core/skill work (e.g., handstand prep).	Peak power output and recovery speed indexes.

3. Core Weekly Microcycle & Workout Logic

The workout delivery system operates on a recurring 7-day microcycle structure configured as follows:

- **Day 1 (Push Day):** Targets chest, anterior deltoids, and triceps.
- **Day 2 (Pull Day):** Targets latissimus dorsi, rhomboids, biceps, and posterior deltoids.
- **Day 3 (Legs & Core Day):** Targets quadriceps, hamstrings, glutes, and rectus abdominis.
- **Day 4:** Full System Passive Rest.
- **Day 5 (Upper Body Power):** Combines compound upper movements with reduced rest timers.
- **Day 6:** Active Core and Isometric holds.
- **Day 7 (Flexibility / Active Recovery):** Targeted full-body dynamic stretching engine.

4. Exercise Database & Tiered Difficulty Matrices

To support both beginner and intermediate levels dynamically within your app, the backend logic should map the following progressive movement tiers. The UI should dynamically display the alternative based on user configuration settings.

Movement Pattern	Beginner Level Tier	Intermediate Level Tier
Horizontal Push	Incline Push-ups (Hands on chair / elevated)	Standard Floor Push-ups / Decline Push-ups
Vertical Push	Pike Push-ups (Feet on ground)	Elevated Pike Push-ups / Wall Handstand Holds
Vertical Pull	Resistance Band Lat Pull-downs / Dead Hangs	Standard Pronated Pull-ups / Chin-ups
Horizontal Pull	Inverted Rows (Under a sturdy table or desk)	Feet-Elevated Inverted Rows / L-sit Rows
Anterior Chain Legs	Bodyweight Air Squats to a chair bench	Unilateral Bulgarian Split Squats
Posterior Chain Legs	Glute Bridges (Both feet flat on floor)	Single-Leg Glute Bridges / Nordic Hamstring Drops
Core Isolation	Standard Forearm Plank (Max hold under 45s)	Hanging Knee Raises / L-Sit Progressions

5. Wellness App UI/UX and Feature Blueprint

To guarantee high engagement, churn mitigation, and pristine usability, integrate these functional blocks:

A. Interactive Workout Player Module

- **Adaptive Timer Engine:** Audio prompts for countdowns. Dynamic switching for regular rest counts (60s) versus power rest windows (90s).
- **Form Validation Loops:** Loop standard 5-10 second looping GIFs/videos showing structural target alignment.

B. Smart Progression Engine & Personalization

- **Dynamic Swapping:** Include an in-app button reading 'Too Hard' or 'Too Easy' allowing an automated real-time alternative from the exercise matrix tier without corrupting session logs.
- **Automated Performance Audits:** Every 30 days, trigger a milestone assessment tracking max repetition capability to dynamically auto-advance tiers.

6. Database Architecture & JSON Schema

The backend structure requires explicit workout object hierarchies. Implement the template layout configuration illustrated in the structural JSON definition below to populate app endpoints:

```
{
  "programId": "calisthenics_90_day_core",
  "version": "1.0.0",
  "block": 1,
  "week": 1,
  "day": 1,
  "routineName": "Foundational Push Day",
  "recommendedRestSeconds": 60,
  "exercises": [
    {
      "exerciseId": "horiz_push_01",
      "name": "Elevated / Incline Push-Up",
      "targetSets": 4,
      "targetReps": "12-15",
      "tempo": "3-0-1-0",
      "alternativeExerciseId": "horiz_push_02"
    }
  ]
}
```